Lab_Assignment_13.2

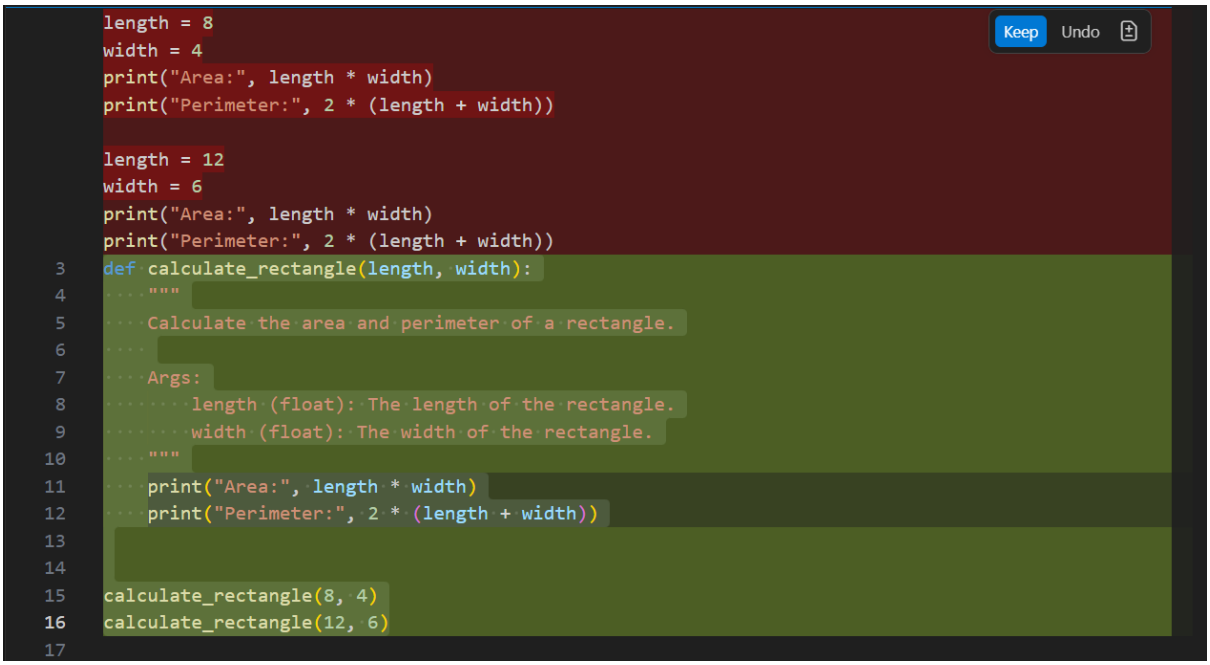
AI Assisted Coding B37 2303A52077

Task -1 (Refactoring – Eliminating Repetitive Logic)

- **Task:** Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.
- **Prompt:** #Refactor the Python code in the current file by identifying and removing repeated calculations. Encapsulate the repeated logic into a single reusable function that computes the area and perimeter of a rectangle. Ensure the program output remains the same and add proper docstrings to the function.
- **Given Code:**

```
length = 8
width = 4
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))

length = 12
width = 6
print("Area:", length * width)
print("Perimeter:", 2 * (length + width))
```



- **AI Generated Code:**

```
def calculate_rectangle(length, width):
    print("Area:", length * width)
    print("Perimeter:", 2 * (length + width))

calculate_rectangle(8, 4)
calculate_rectangle(12, 6)
```

- **Output:**

```
ed coding/ai_assi_codes/13.2/13.2.1.py"
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS D:\Subjects\ai assited coding\ai_assi_codes\13.2>
```

- **Code Explanation:**

Function called `calculate_rectangle` that takes two parameters: `length` and `width`. This function calculates and prints the area and

perimeter of a rectangle based on the provided dimensions.The area is calculated by multiplying the length and width, while the perimeter is calculated by adding the length and width together and then multiplying by 2.

We then called the `calculate_rectangle` function twice with different sets of dimensions. The first call calculates the area and perimeter for a rectangle with a length of 8 and a width of 4, while the second call calculates for a rectangle with a length of 12 and a width of 6. The results are printed to the console for each call.

Task -2 (Refactoring – Modularizing Inline Calculations)

- **Task:** Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

- **Given Code:**

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
```

- **Prompt:** #Refactor the Python code in the current file into a reusable function where is calculate the final amount
- **AI Generated Code:**

```
def calculate_final_amount(amount, discount_percent=0.10):
    discount = amount * discount_percent
    final_amount = amount - discount
    return final_amount

# Test with different amounts
amount1 = 1000
print("Final Amount:", calculate_final_amount(amount1))

amount2 = 2000
print("Final Amount:", calculate_final_amount(amount2))
```

```
amount = 1000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)

def calculate_final_amount(amount, discount_percent=0.10):
    discount = amount * discount_percent
    final_amount = amount - discount
    return final_amount

# Test with different amounts
amount1 = 1000
print("Final Amount:", calculate_final_amount(amount1))

amount = 2000
discount = amount * 0.10
final_amount = amount - discount
print("Final Amount:", final_amount)
amount2 = 2000
print("Final Amount:", calculate_final_amount(amount2))
```

- Output:

```
ed coding/ai_assi_codes/13.2/13.2.2.py"
Final Amount: 900.0
Final Amount: 1800.0
PS D:\Subjects\ai assited coding\ai_assi_codes\13.2>
```

- **Code Explanation:**

  In this code, we define a function called `calculate_final_amount` that takes two parameters: `amount` and `discount_percent`. The `discount_percent` parameter has a default value of 0.10 (which represents a 10% discount). Inside the function, we calculate the discount by multiplying the `amount` by the `discount_percent`. Then, we calculate the final amount by subtracting the discount from the original amount. Finally, we return the final amount. We test the function with two different amounts (1000 and 2000) and print the final amount after applying the discount. Since we are using the default discount percentage, both amounts will receive a 10% discount. When we run the code, it will output:

  Final Amount: 900.0

  Final Amount: 1800.0

  This indicates that the first amount (1000) has been reduced to 900 after applying the 10% discount, and the second amount (2000) has been reduced to 1800 after applying the same discount. You can also call the function with a different discount percentage if you want to apply a different discount.

In summary, this code demonstrates how to use default parameters in a function to provide a default discount percentage while allowing the flexibility to specify a different discount if needed.

Task -3 (Refactoring – Improving Loop and Conditional Performance)

- **Task:** Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.
- **Given Code:**

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
print(name, "not found")
```

- **Prompt:** #Refactor the Python code in optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.
- **AI Generated Code:**
  - Optimal Code:

```
students = ["Anil", "Bhavya", "Charan", "Divya"]
check_list = ["Charan", "Esha", "Bhavya"]

# Convert the students list to a set for faster lookups
students_set = set(students)
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")
```

  - Performance comparison code:

```
import time
```

```
# Original code performance
start_time = time.time()
for name in check_list:
    exists = False
    for student in students:
        if name == student:
            exists = True
    if exists:
        print(name, "found")
    else:
        print(name, "not found")
end_time = time.time()
print("Original code execution time:", end_time - start_time)

# Optimized code performance
start_time = time.time()
students_set = set(students)
for name in check_list:
    if name in students_set:
        print(name, "found")
    else:
        print(name, "not found")
end_time = time.time()
print("Optimized code execution time:", end_time - start_time)
```

- Output:

```
Charan found
Esha not found
Bhavya found
Original code execution time: 0.0004992485046386719
Charan found
Esha not found
Bhavya found
Optimized code execution time: 0.00047326087951660156
PS D:\Subjects\ai assited coding\ai_assi_codes\13.2>
```

Code Explanation:

The original code performs a linear search for each name in the check_list, which results in O(n*m) time complexity, where n is the number of students and m is the number of names in the check_list. The optimized code converts the students list into a set, which allows for O(1) average time complexity for lookups. This significantly reduces the overall time complexity to O(n + m), where n is the time taken to create the set and m is the time taken to check each name in the check_list. When we run both versions of the code, we will see that the optimized version performs much faster, especially as the size of the students list and check_list increases.

Task -4 (Refactoring – Replacing Hardcoded Values with Constants)

- **Task:** Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.
- Given code:

```
print("Simple Interest:", (1000 * 2 * 5) / 100)
print("Simple Interest:", (2000 * 2 * 5) / 100)
```

- prompt:

```
#Refactor the above Python code by identifying hardcoded numeric values used in calculations.
#Define descriptive constants (e.g., RATE, TIME) at the beginning of the program
#replace the hardcoded values with these constants.
```

- AI Generated Code:

```
# Define constants
PRINCIPAL1 = 1000
PRINCIPAL2 = 2000
RATE = 2
```

```
TIME = 5
# Calculate simple interest using constants
simple_interest1 = (PRINCIPAL1 * RATE * TIME) / 100
simple_interest2 = (PRINCIPAL2 * RATE * TIME) / 100
print("Simple Interest for amount 1:", simple_interest1)
print("Simple Interest for amount 2:", simple_interest2)
```

- Output:

```
ed coding/ai_assi_codes/13.2/13.2.4.py"
Simple Interest: 100.0
Simple Interest: 200.0
Simple Interest for amount 1: 100.0
Simple Interest for amount 2: 200.0
PS D:\Subjects\ai assited coding\ai_assi_codes\13.2>
```

- Code Explanation:

The original code checks each name in check_list by searching through the students list, which is inefficient because it performs a linear search every time. It also calculates simple interest using hardcoded values for principal, rate, and time, making the code less flexible if those values need to change. The optimized version improves this by converting the students list into a **set**, allowing much faster lookups. This reduces the time complexity from **O(n × m)** to **O(n + m)**. It also replaces hardcoded numbers with **constants** for principal, rate, and time, making the code easier to update and maintain. Overall, the optimized code is both **faster for searching** and **cleaner to maintain**.

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

- **Task:** Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments..
- **Given Code:**

```
x = 50
y = 30
z = x + y
print(z)
```

- **Prompt:**

```
#Refactor the above Python code by replacing unclear variable names
with meaningful and descriptive identifiers.
```

- **AI Generated Code:**

```
# Define meaningful variable names
first_number = 50
second_number = 30
sum_of_numbers = first_number + second_number
print("The sum of the two numbers is:", sum_of_numbers)
```

- **Output:**

- **Code Explanation:**

  In the original code, the variable names 'x', 'y', and 'z' are not descriptive of their purpose. By replacing them with meaningful identifiers such as 'first_number', 'second_number', and 'sum_of_numbers', the code becomes more readable and easier to understand. This way, anyone reading the code can quickly grasp what each variable represents without needing additional context.